餐饮门店 · AI 智能经营助手

# 小满

连接餐厅已有的经营数据，在现有经营系统之上增加一层 AI 能力。

解决两件事：**不断货，不漏单**。

01
**让现有系统负责记录**

02
**让 AI 负责理解**

03
**让老板负责决策**

智能补货　库存风险预测　经营分析　AI 店长　AI 前台

# ERP × 小满：谁记录，谁判断

| | ERP · 经营数据底座 | 小满 · AI 智能经营层 |
| --- | --- | --- |
| 定位 | 记录已发生的业务事实 | 理解事实、做出判断 |
| 负责 | 销售、库存、采购、报损、供应商、菜品配方的准确记录与留存 | 读取这些数据 → 分析 → 预测 → 生成可执行的建议 |
| 回答 | 「现在库存多少？最近卖了多少？采购了多少？」 | 「明天可能卖多少？什么时候可能缺货？应该采购多少？」 |
| 输出 | 准确、可追溯、可核对的经营数据 | 老板一看就懂、可直接确认或调整的决策建议 |

# 智能补货：一个完整的经营闭环




**小满的建议长这样**

| | | |
|---|---|---|
| **牛肉** | 建议采购 **+15 kg** | 供应商甲 · 周三配送 · 次日达 |
| **生菜** | 建议采购 **+8 kg** | 供应商乙 · 每日 22:00 前下单 |
| **虾滑** | 库存偏高 · 暂缓 | 供应商丙 · 起订量 10 箱，未达量 |

依据：近 4 周周六销量 · 已确认预订 12 桌 · 在途未到货 · 供应商配送周期

**建议要先过这几关**

- **供应商** 报价 · 账期 · 供货是否稳定
- **供货方式** 配送日 · 起订量 · 到货时效
- **在途库存** 已下单未到货的，不重复买
- **库容保质** 放不下、放不久的，少买

# 整体架构：五层各管一件事

**角色交互层**　人在这里提问、看建议、拍板　老板 · 店长 · 客户

↓ 人提问　↑ AI 返回建议

**AI 能力层**

**一个 AI 底座，服务所有角色**

AI 调度器　统一 AI 底座

全系统只有一套，角色差异放在配置里

↑ 整理好的经营数据

**系统层**　把现有系统的数据整理成一套口径　数据接入 · 归一化 · 清洗

**数据源层**　现有系统照常跑，数据从这儿来　联锁云 ERP · 美团外卖 · 收银 POS · 供应商 / 预订

# AI 系统设计：全公司只有一个 AI



# AI 店长：老板直接问，AI 直接答

小满 · 经营助手

这个月毛利怎么掉了？

小 满

本月毛利率 **52.1%**，比上月下降 **3.2pt**。主要原因：牛肉采购价上涨 8%，以及报损率从 2.1% 升到 3.4%（集中在叶菜）。

周六要不要多备点牛肉？

小 满

建议 **+15 kg**。依据：近 4 周周六牛肉销量均值 18.6kg，本周六已有 12 桌预订（含 3 桌牛肉锅），天气转凉带动锅类销量上行。

**经营分析**

营业额、毛利、菜品结构、报损，一句话问出结论。

**库存风险预警**

提前告知哪道菜、哪个原料快卖断。

**补货建议**

给出品类、数量、到货时间与判断依据。

**所有回答都基于真实经营数据。**小满只读联锁云里已发生的销售、库存、采购与报损，不猜测、不虚构。

# AI 前台：把每一次咨询接住，也把需求变成数据

小满 · 顾客咨询

明晚 7 点，8 个人还有位置吗？

小 满

明晚 19:00 还有位置，已为您预留 **8 人桌**。需要加一张儿童座椅吗？

要的，另外你们招牌牛肉还有吗？

小 满

有的。已备注 **儿童座椅 1 张**。要不要提前为您预留一份招牌牛肉锅？量有限，先备着更稳妥。

**预订接待**

人数、时间、忌口一次问清

**菜品咨询**

招牌菜、口味、过敏原秒回

**排队与等位**

告知等待时间，缓解高峰压力

**推荐与加购**

结合库存推荐更合适的菜

这轮对话产生了 **1 条预订 + 1 条菜品意向** —— 回流成经营数据，计入明天的销量预测。

# 实施与数据接入：不改系统，先把数据用起来

## 数据接入 · 可适配的三种情况

**情况一** **数据已在现有系统中**

通过客户现有系统 / 联锁云读取，无需额外对接。

**情况二** **有系统，但接口不好接**

先定期导出表格对接，跑通业务再优化。

**情况三** **数据源不在现有系统内**

由小满直接接入对应数据源（如美团、收银、手工录入）。

## 实施路径 · 从单点场景切入

1. **接入数据**
   打通销售、库存、采购等基础经营数据
2. **理解数据**
   建立菜品、配方、供应商与单位的对应关系
3. **跑通场景**
   从「智能补货」单点切入，先解决不断货
4. **扩展能力**
   逐步扩展到经营分析、AI 店长与 AI 前台

# 技术选型：都用成熟的东西，不从零造轮子

## Node.js

服务端 · 数据 / 工具 / 权限 / 接入

- 与前端同一套语言，接口、权限、数据口径在一个工程里对齐
- 数据接入、工具服务、权限系统一套人马维护，不用养两支队伍
- 生态最成熟的运行时之一，招得到人、查得到资料

一套人马就能维护

## pi

AI 层 · Agent Harness，跑在服务端里

- 不自研 AI 框架，基于成熟的 pi 做二次开发
- 小满只补业务定制：工具清单 · 角色权限 · 知识库
- 模型按效果与成本随时替换，不绑死任何供应商

创新只发生在这一层

## uni-app

界面端 · 小程序 / H5 / App

- 一次开发、多端运行，微信小程序、H5、App 都覆盖
- 与服务端同一套 JS 体系，前后端衔接没有切换成本
- 生态成熟，主流平台全部支持

一套代码，多端就绪

工程承诺 —— **成熟技术打底，创新只发生在 AI 那一层。**

# 报价：MVP 一次性交付

MVP一次性交付价

¥8 万

交付周期

两个月

交付内容

- **智能补货闭环** — 需求预测 → 库存风险判断 → 补货建议 → 老板确认 → 采购执行记录
- **客户咨询闭环** — 顾客咨询 → AI 回复 → 执行动作 → 系统记录
- **老板端** — 微信小程序 / H5，一次开发、多端运行
- **数据接入与归一化** — 数据 · 清洗 · 结构化 · 口径统一

计价依据 · 中国软件行业协会《软件开发和服务项目价格构成及评估方法》

**一句话：两个月（约 45 个工作日）的工作量，折算成 8 万。**

**软件开发价格 = 开发工作量 × 开发费用 / 人月** | **开发工作量 = 估算工作量经验值 × 风险系数 σ × 复用系数 τ**

估算工作量经验值 = 两个月 ≈ 45 人天 ÷ 21.75 ≈ 2.07 人月 | σ = 1.5（接入与 AI 效果的不确定性） | τ = 0.9（复用 pi 骨架与成熟技术，不做重复开发）

开发工作量 = 2.07 × 1.5 × 0.9 ≈ 2.79 人月 | 开发费用 / 人月 = 20,000 × 1.45 ≈ 29,000 元

软件开发价格 = 2.79 × 29,000 ≈ **8.0 万元**